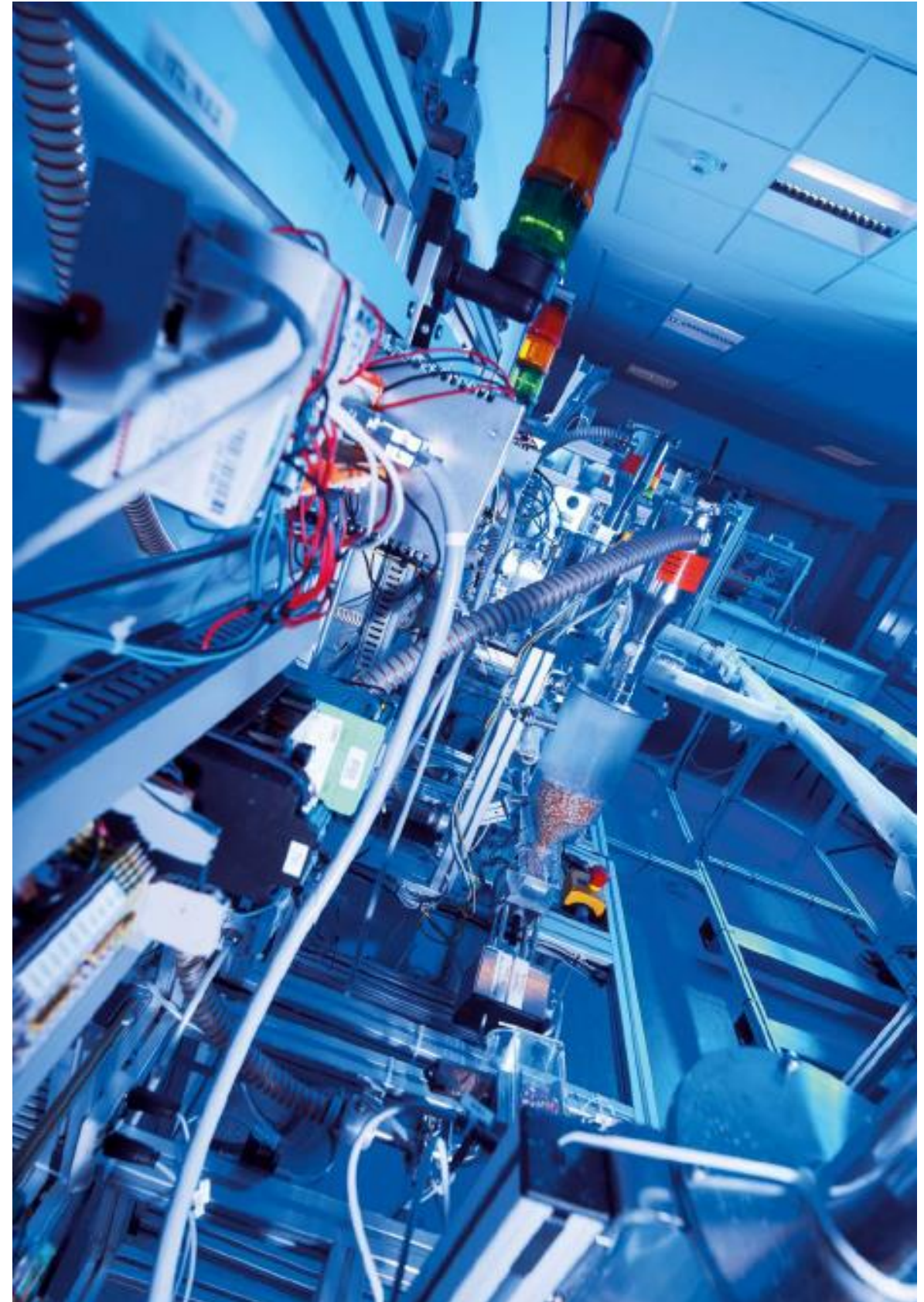


# Echtzeit-Datenverarbeitung

## C-Programmierung (Recap)

**Prof. Dr. Henning Trsek**

Technische Hochschule Ostwestfalen-Lippe  
Fachbereich Elektrotechnik und Technische Informatik  
inIT - Institut für industrielle Informationstechnik  
[henning.trsek@th-owl.de](mailto:henning.trsek@th-owl.de)

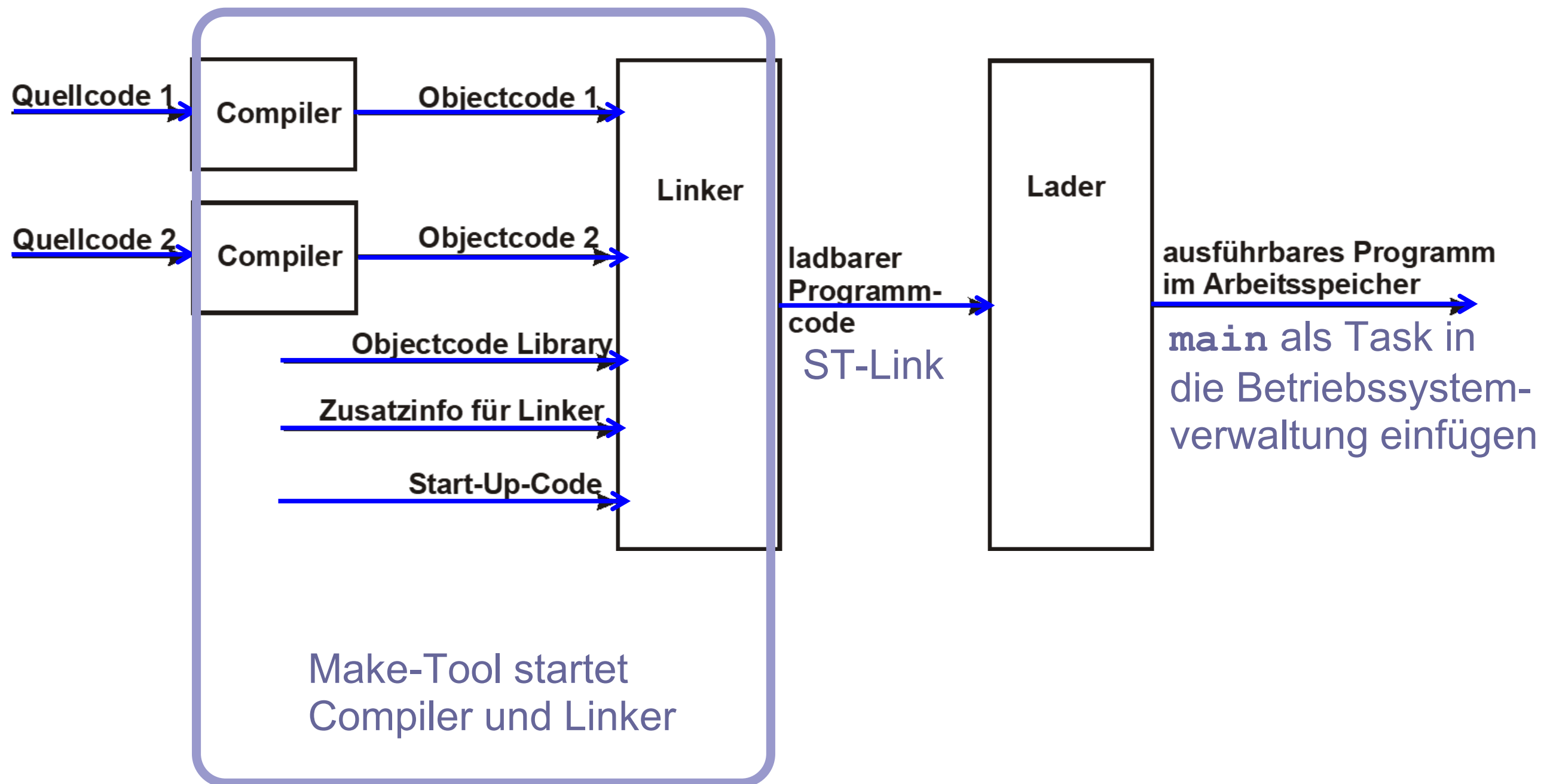


# Überblick Peripherie

---

- ▶ C Programmierung (Wiederholung)
  - Datentypen
  - Operatoren
  - Zeiger

# Übersetzen von C-Programmen



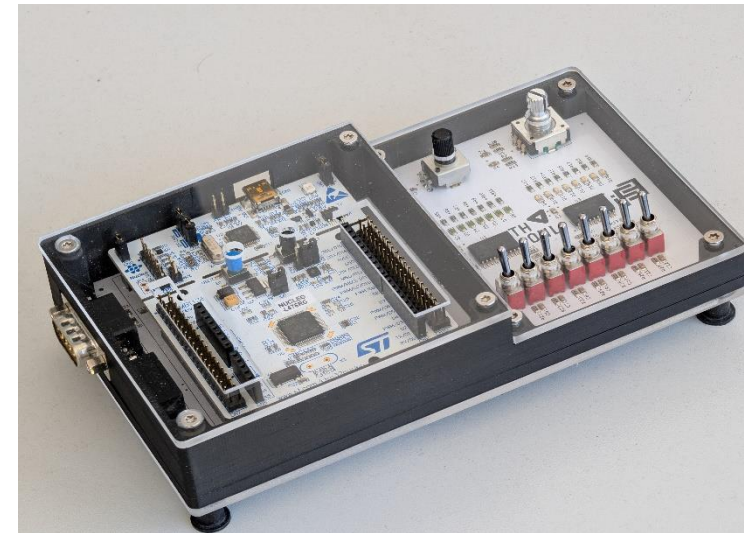
# Software-Tools

---



Betriebssystem  
(nicht echtzeitfähig)

- Editor
- Compiler
- Linker
- Make-Tool
- Terminal-Emulation



Betriebssystem  
(echtzeitfähig)

- Lader des Echtzeit-  
betriebssystems

# Einfaches Task-Beispiel

```
#include <stdio.h>
...
// --- constants -----
#define VALUE 5
#define MAX 10
// --- prototypes -----
void main(void);
// --- modul-variables -----
int count;
int i;
// --- start of TASK -----
void test(void) {
    count = 0;
    for (i=0; i<MAX; i++)
    {
        count = count + VALUE;
        printf("test %i", count);
    }
    vTaskDelete()
}
```

← Bibliotheksfunktionen

← Definition von Konstanten

← Bekanntgabe von Funktionen und Variablentypen

← Erzeugen von Variablen (Speicherplatz bereitstellen)

← Task

**freeRTOS Tasks müssen eine Endlosschleife beinhalten oder beendet werden**

Ausgabe auf Standardschnittstelle

# Datentypen

---

char , signed char  
unsigned char

int

unsigned int

short int

unsigned short int

long int

unsigned long int

float

double

Größe im Speicher:

1 Byte

Vergleich mit Assembler:

char : **BYTE**

Definition und Wertzuweisung von Character-Variablen

```
char led_reihe_oben;  
char led_reihe_unten;  
...  
led_reihe_oben = 0x55;  
led_reihe_unten = 0x55;
```

Ausgabe der Variablen auf  
die Hardware

-> Pointer

(jede zweite LED leuchtet)

# Datentypen

char , signed char  
unsigned char  
int  
unsigned int  
short int  
unsigned short int  
long int  
unsigned long int  
float  
double

Größe im Speicher:

**int** : ? (2 oder 4 Byte)

**short int** : 2 Byte

**long int** : 4 Byte

Vergleich mit Assembler:

**short int** : **WORD**

**long int** : **LONG**

Definition und Wertzuweisung von Integer-Variablen

```
long int    var1;  
short int   var2;  
...  
var1 = 0x7FFFFFFF;  
var2 = 0x7FFF;
```

```
char  schalter;  
int    var3;  
...  
var3 = (int)schalter;  
var1 = (long int)var2;
```

Cast-Operator  
Datentypwandlung  
*immer verwenden !!!*

# Datentypen

---

char , signed char  
unsigned char  
int  
unsigned int  
short int  
unsigned short int  
long int  
unsigned long int  
float  
double

Größe im Speicher:

**float** : 4 Byte

**double** : 8 Byte

Definition und Wertzuweisung von Float-Variablen

```
float inf2 f1;
```

```
float inf2_f1 = 3.1415927;
```

```
...
```

```
inf2_f1 = 3.1415927;
```

# Datentypen

---

Typkonvertierung : **cast**-Operator

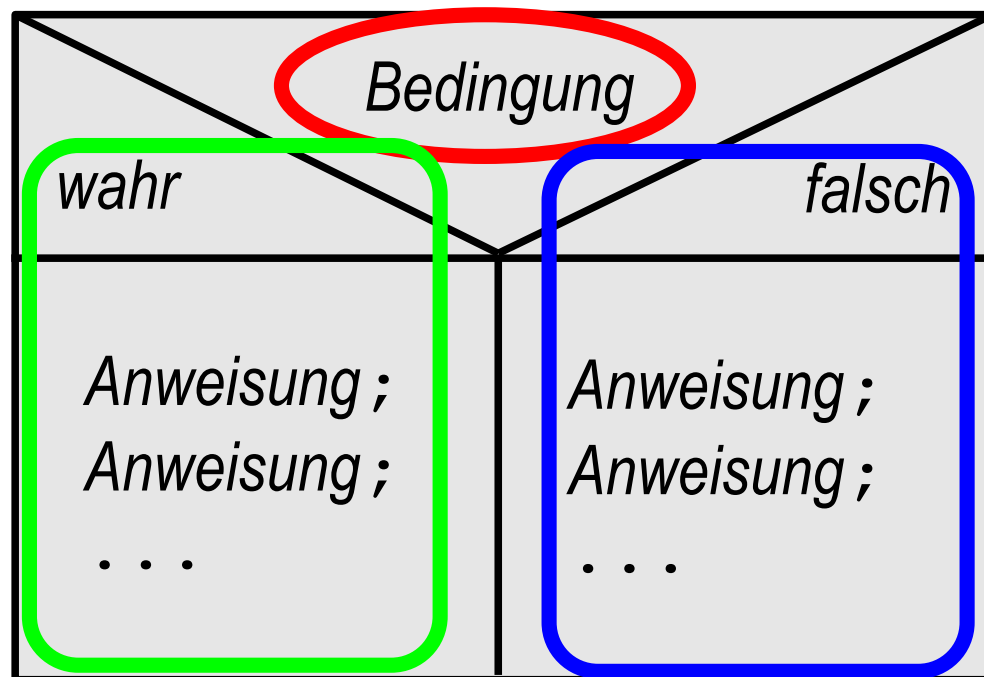
Unterschiedlicher Aufwand:

```
...  
char      schalter_8bit;  
short int index;  
float      summe;  
...  
schalter_8bit = 0x03;  
index = (short int) schalter_8bit;  
summe = (float) index;  
...
```

Vorzeichenrichtige Erweiterung  
(0x03 -> 0x0003)

Umrechnung in anderes Intern-Format  
(0x0003 -> 0x40400000)

# Verzweigung im Programmablauf



```
if (Bedingung)
{
    Anweisung ;
    Anweisung ;
    ...
}
else
{
    Anweisung ;
    Anweisung ;
    ...
}
```

optional

# Operatoren

---

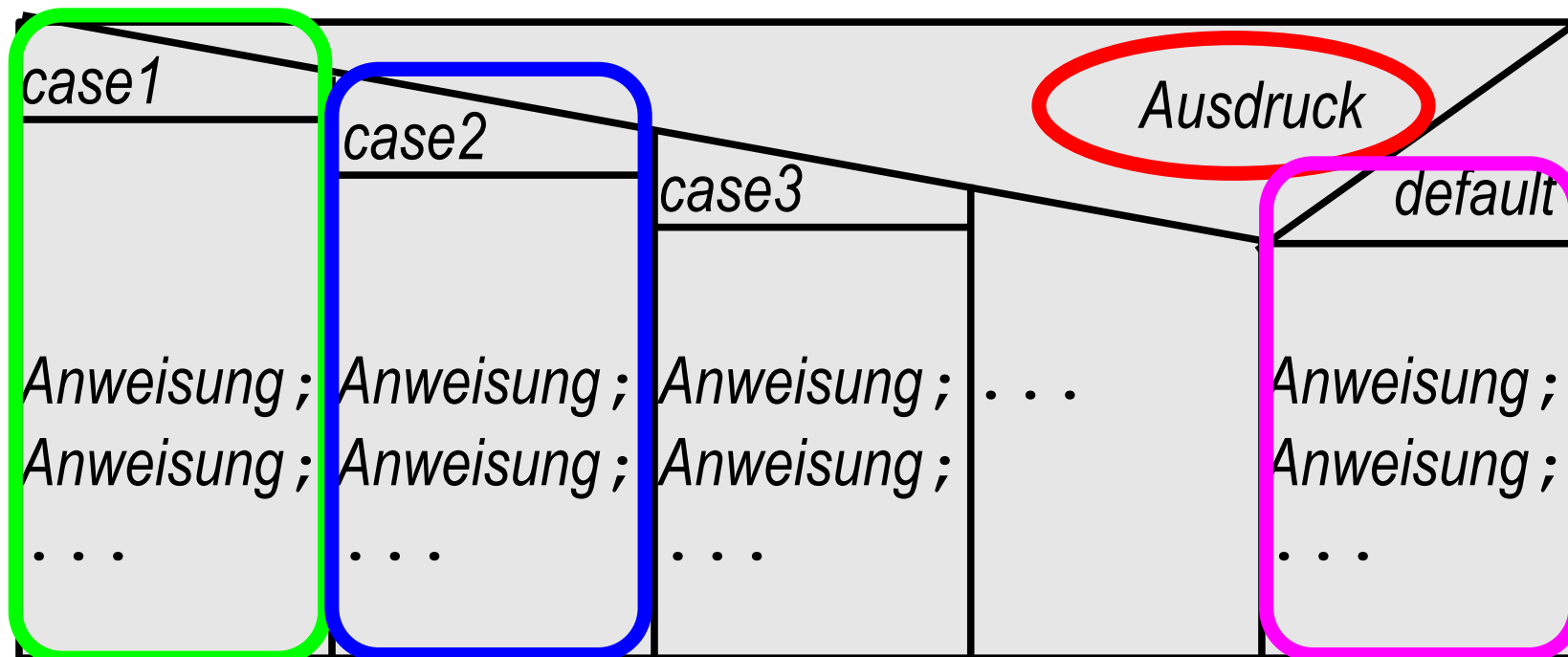
## Vergleichsoperatoren

|    |                                       |
|----|---------------------------------------|
| >  | größer als<br>(greater then)          |
| >= | größer oder gleich<br>(greater equal) |
| <  | kleiner als<br>(less then)            |
| <= | kleiner oder gleich<br>(less equal)   |
| == | <b>Vorsicht !</b> gleich<br>(equal)   |
| != | ungleich<br>(not equal)               |

## Logische Operatoren

|    |                                                                                   |
|----|-----------------------------------------------------------------------------------|
| && | logisches UND<br>(wenn Bedingung1 <b>und gleichzeitig</b> Bedingung2 erfüllt ist) |
|    | logisches ODER<br>(wenn Bedingung1 <b>oder</b> Bedingung2 erfüllt ist)            |
| !  | logisches NICHT<br>(wenn die Bedingung <b>nicht</b> erfüllt ist)                  |

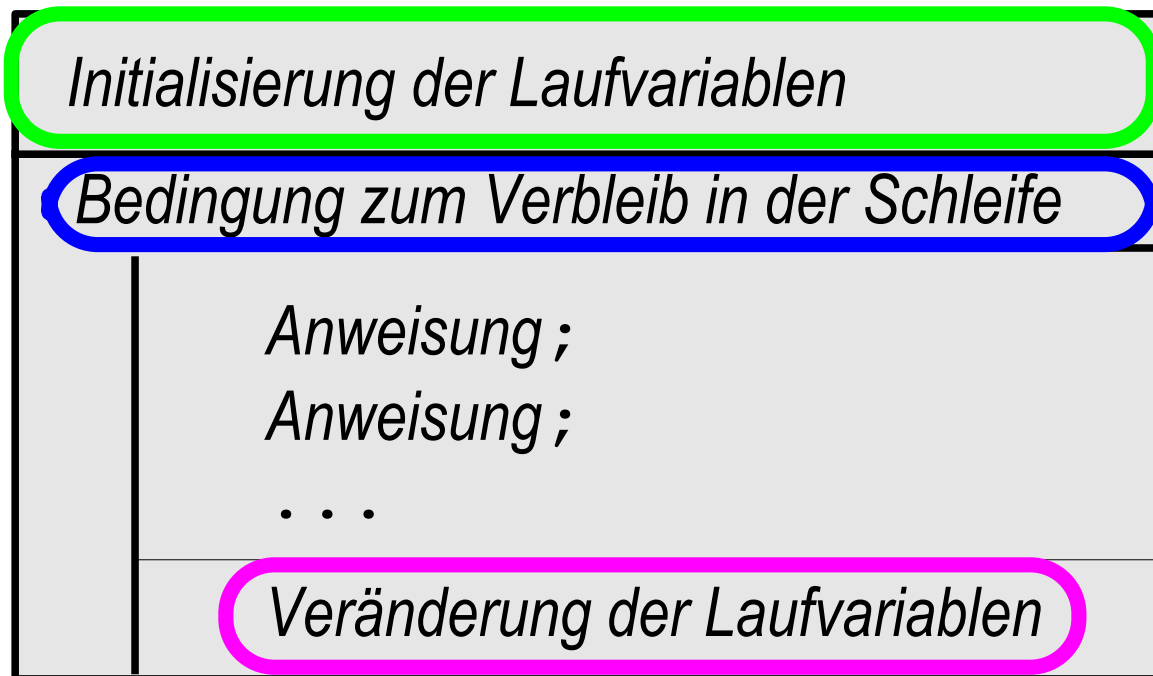
# Mehrfach-Alternative



```
switch (Ausdruck)
{
    case KONSTANTE1:
        Anweisung ;
        Anweisung ;
        ...
        break;
    case KONSTANTE2:
        Anweisung ;
        Anweisung ;
        ...
        break;
    case KONSTANTE3:
        Anweisung ;
        Anweisung ;
        ...
        break;
    default:
        Anweisung ;
        Anweisung ;
        ...
        break;
}
```

# Schleife im Programmablauf

---



```
for (Initialisierung der Laufvariablen; Bedingung zum Verbleib in der Schleife; Veränderung der Laufvariablen)
{
    Anweisung ;
    Anweisung ;
    . . .
}
```

# Schleife im Programmablauf

---

Beispiele:

**Variablendefinition vor der Schleife !**

```
int count;  
...  
for (count=0; count< 10; count++)  
{  
    Anweisung ;  
    Anweisung ;  
    ...  
}
```

# Schleife im Programmablauf

---

Beispiele:

```
int count;  
...  
for (count=0; count< 10; count++)  
{  
    Anweisung ;  
    Anweisung ;  
    ...  
}
```

```
for ( ; ; ) ← Endlosschleife !  
{  
    Anweisung ;  
    Anweisung ;  
    ...  
}
```

Vorsicht bei Tasks mit hoher Priorität.

# Schleife im Programmablauf

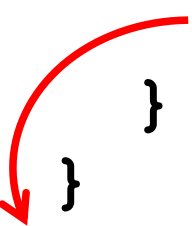
---

Beispiele:

```
int count;  
...  
for (count=0; count< 10; count++)  
{  
    Anweisung ;  
    Anweisung ;  
    ...  
}
```

```
for (;;)   
{  
    Anweisung ;  
    Anweisung ;  
    ...  
}
```

```
for (;;)   
{  
    Anweisung ;  
    Anweisung ;  
    ...  
    if (Bedingung)  
    {  
        break ;  
    }  
}
```



# Zeiger (Pointer Variablen)

---

Ein Zeiger ist eine Variable, die die Adresse eines im Speicher vorhandenen Objektes aufnehmen kann.

Eine Zeigervariable verweist auf die jeweilige Adresse.



Beispiel:

`char *xyz_ptr;` Pointerdefinition:  
Pointer auf char-Objekt  
(4 Byte) (1 Byte)

`char if2;`

Zuweisung: Adresse von `if2` -> Pointer

`xyz_ptr = &if2;` Danach zeigt `xyz_ptr` auf `if2`.

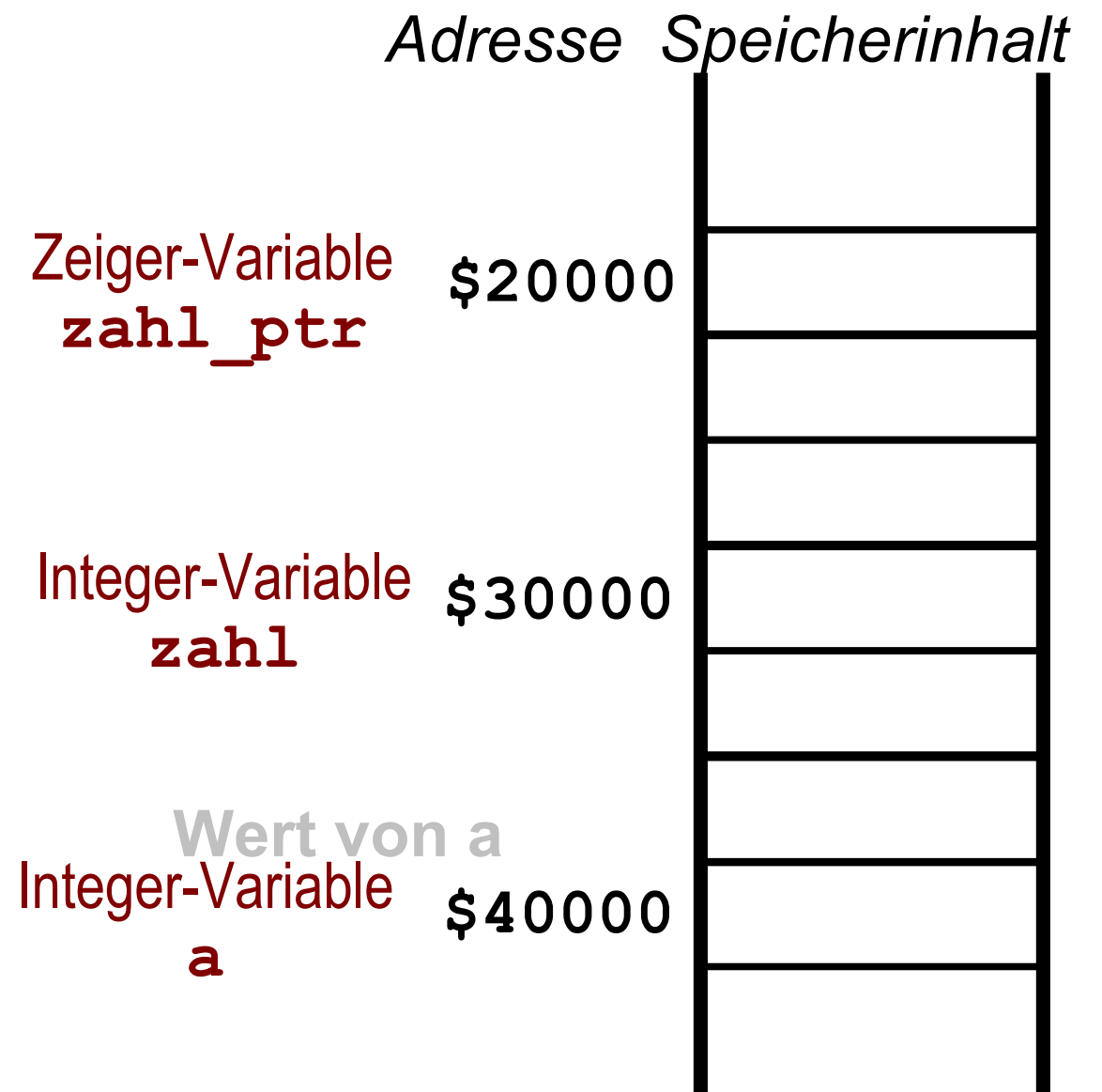
# Zeiger Beispiel 1

```
long int *zahl_ptr;  
long int zahl;  
long int a;
```

```
zahl = 0x0001FE58;  
zahl_ptr = &zahl;  
a = 0;
```

## Zuweisung

```
a = zahl;           $0001FE58  
a = (long int)&zahl; $00030000  
a = (long int)zahl_ptr; $00030000  
a = (long int)&zahl_ptr; $00020000  
a = *zahl_ptr;      $0001FE58
```



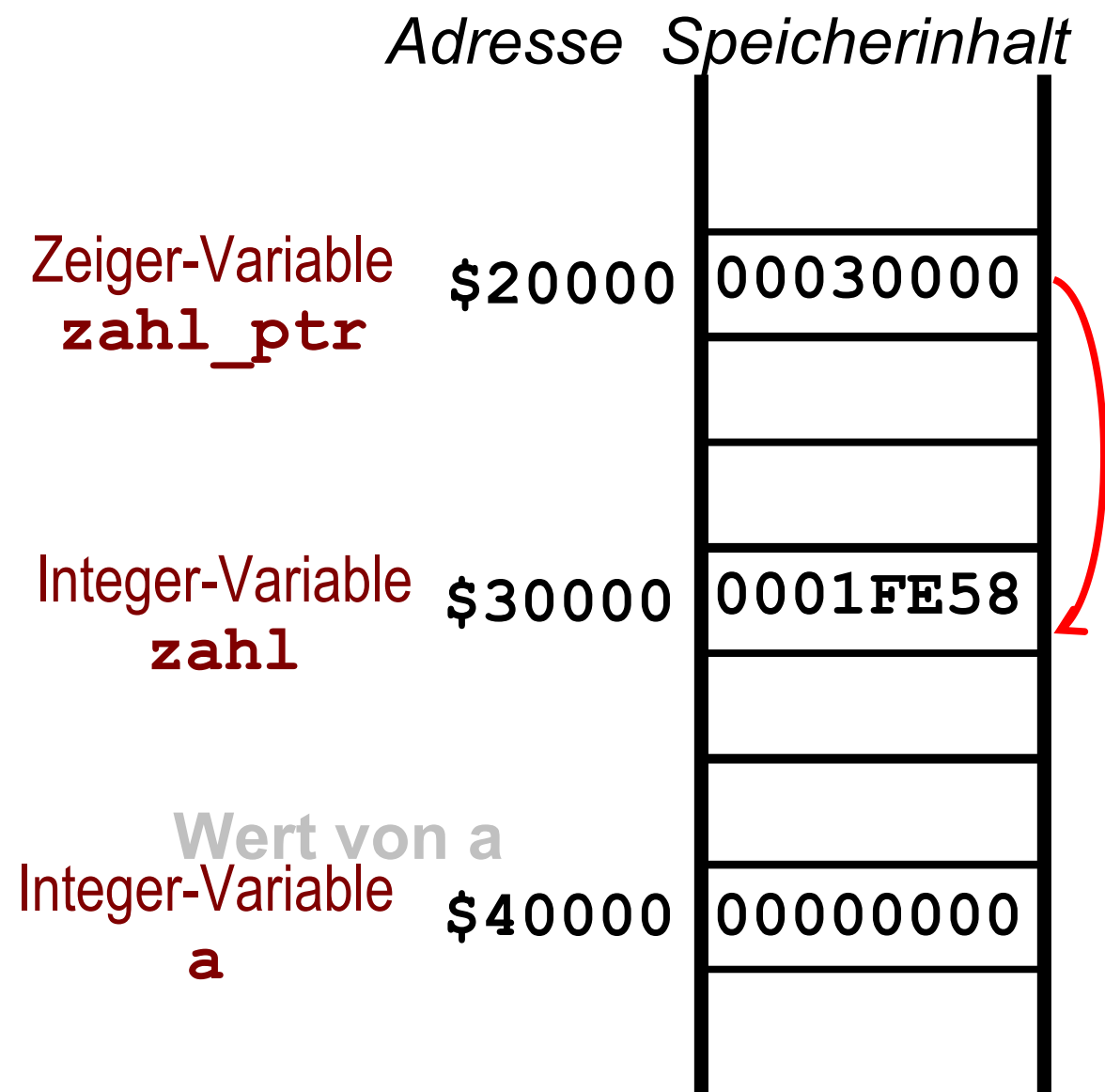
# Zeiger Beispiel 1

```
long int *zahl_ptr;  
long int zahl;  
long int a;
```

```
zahl = 0x0001FE58; ←  
zahl_ptr = &zahl; ←  
a = 0; ←
```

## Zuweisung

```
a = zahl;           $0001FE58  
a = (long int)&zahl; $00030000  
a = (long int)zahl_ptr; $00030000  
a = (long int)&zahl_ptr; $00020000  
a = *zahl_ptr;      $0001FE58
```



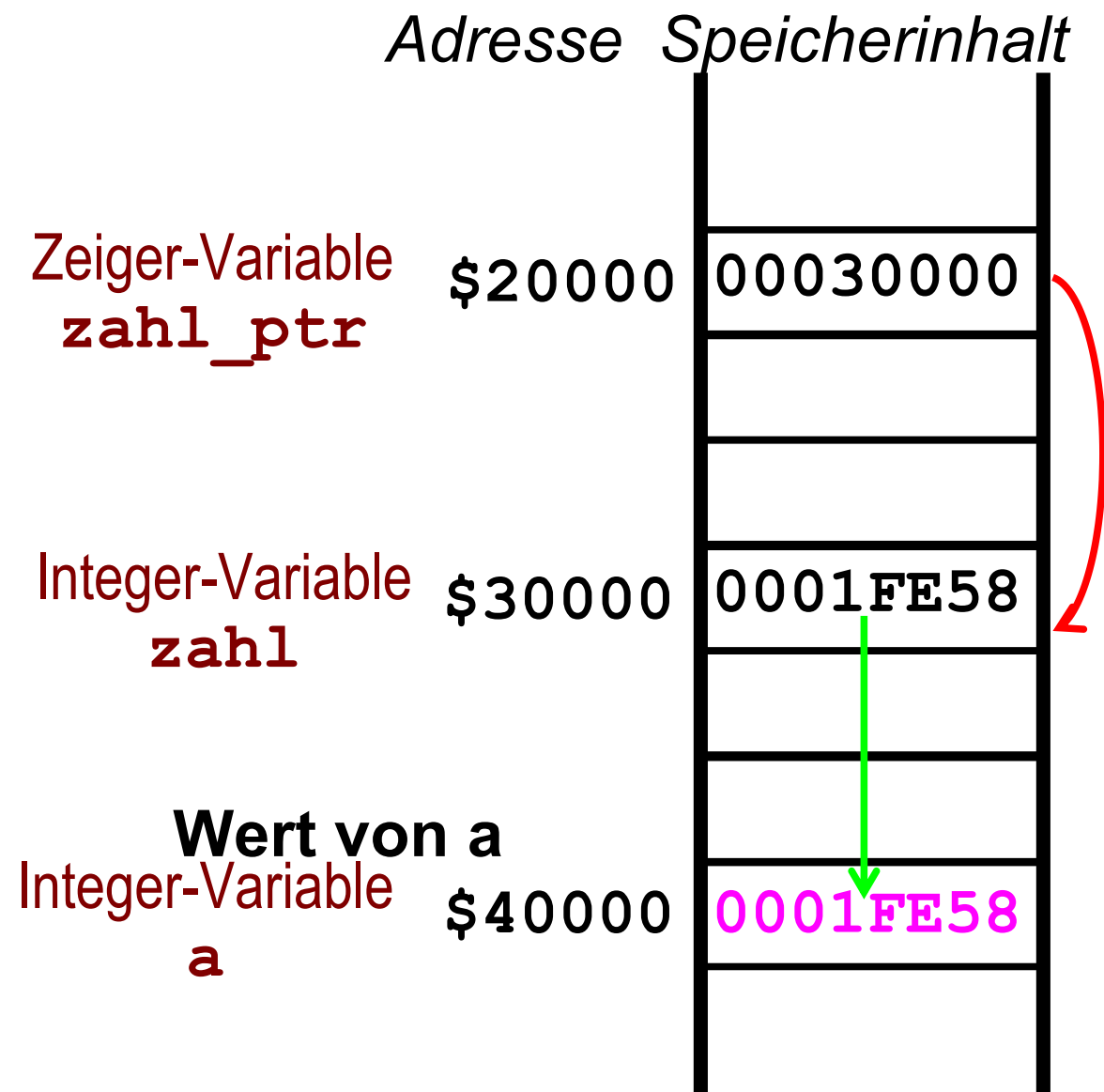
# Zeiger Beispiel 1

```
long int *zahl_ptr;  
long int zahl;  
long int a;
```

```
zahl = 0x0001FE58;  
zahl_ptr = &zahl;  
a = 0;
```

## Zuweisung

```
a = zahl;           $0001FE58  
a = (long int)&zahl;  $00030000  
a = (long int)zahl_ptr; $00030000  
a = (long int)&zahl_ptr; $00020000  
a = *zahl_ptr;       $0001FE58
```



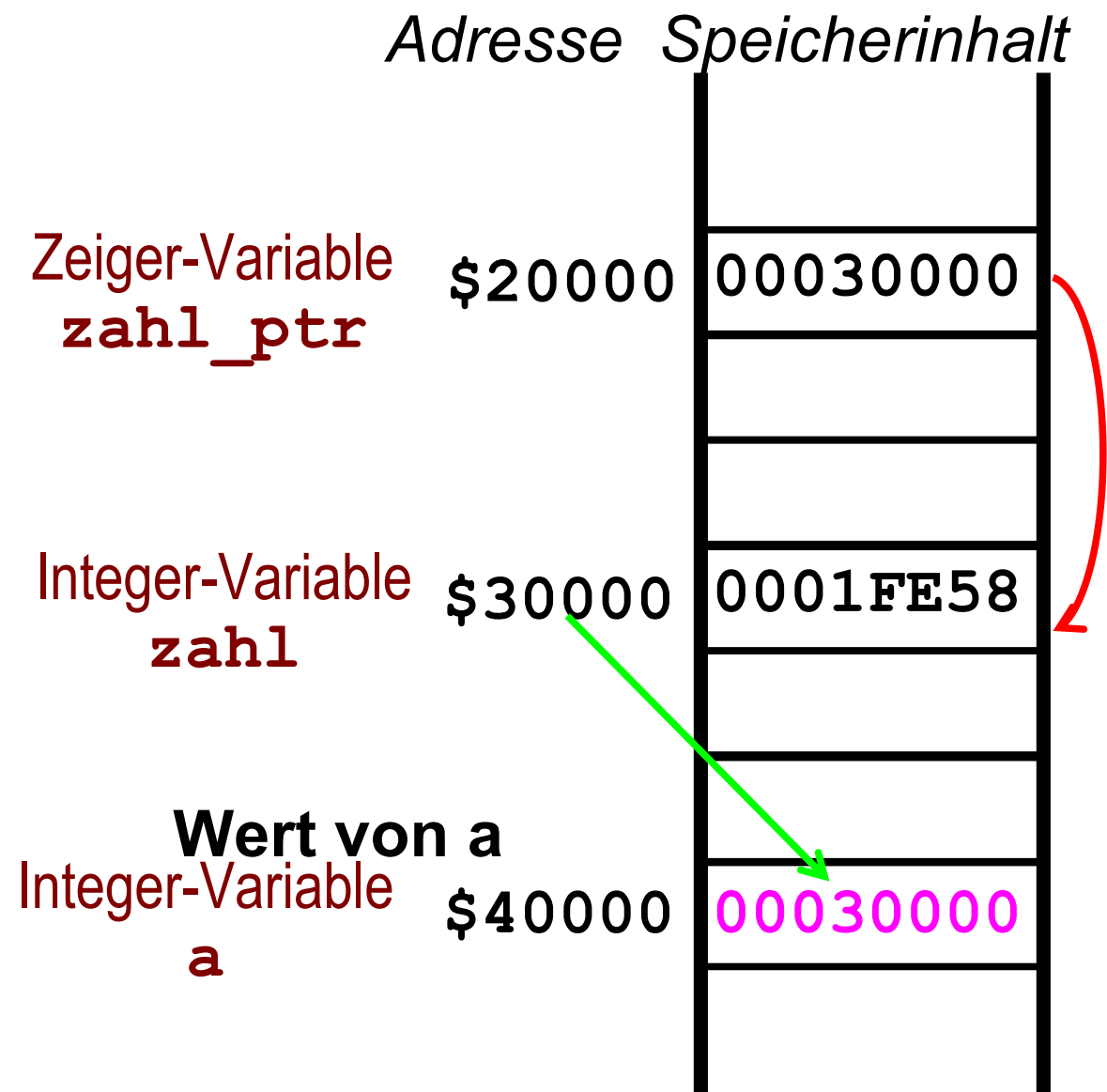
# Zeiger Beispiel 1

```
long int *zahl_ptr;  
long int zahl;  
long int a;
```

```
zahl = 0x0001FE58;  
zahl_ptr = &zahl;  
a = 0;
```

## Zuweisung

```
a = zahl;           $0001FE58  
a = (long int)&zahl;  $00030000  
a = (long int)zahl_ptr; $00030000  
a = (long int)&zahl_ptr; $00020000  
a = *zahl_ptr;       $0001FE58
```



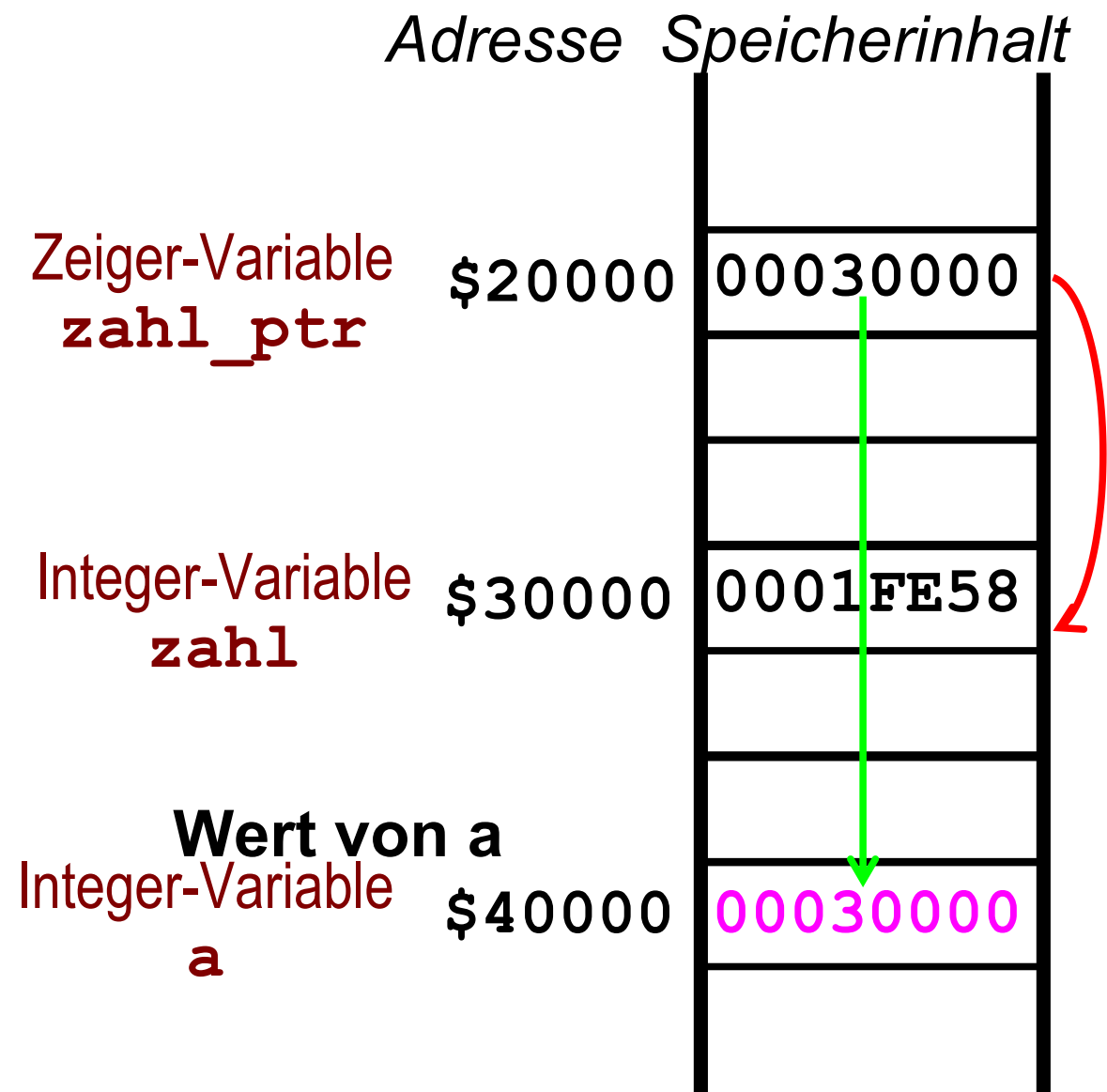
# Zeiger Beispiel 1

```
long int *zahl_ptr;  
long int zahl;  
long int a;
```

```
zahl = 0x0001FE58;  
zahl_ptr = &zahl;  
a = 0;
```

## Zuweisung

```
a = zahl;           $0001FE58  
a = (long int)&zahl; $00030000  
a = (long int)zahl_ptr; $00030000  
a = (long int)&zahl_ptr; $00020000  
a = *zahl_ptr;       $0001FE58
```



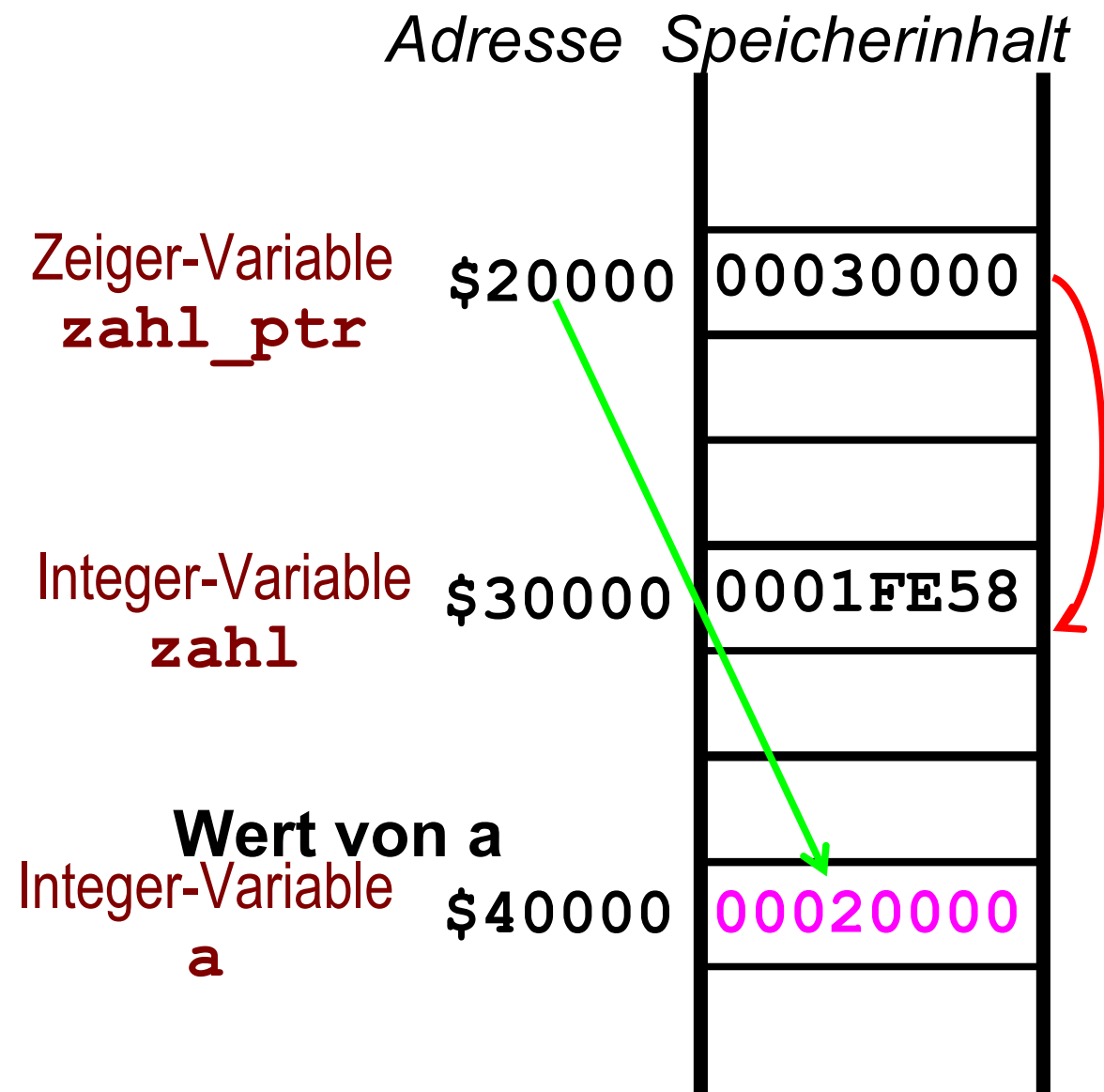
# Zeiger Beispiel 1

```
long int *zahl_ptr;  
long int zahl;  
long int a;
```

```
zahl = 0x0001FE58;  
zahl_ptr = &zahl;  
a = 0;
```

## Zuweisung

|                                                  |                                |
|--------------------------------------------------|--------------------------------|
| <code>a = zahl;</code>                           | <code>\$0001FE58</code>        |
| <code>a = (long int)&amp;zahl;</code>            | <code>\$00030000</code>        |
| <code>a = (long int)zahl_ptr;</code>             | <code>\$00030000</code>        |
| <u><code>a = (long int)&amp;zahl_ptr;</code></u> | <u><code>\$00020000</code></u> |
| <code>a = *zahl_ptr;</code>                      | <code>\$0001FE58</code>        |



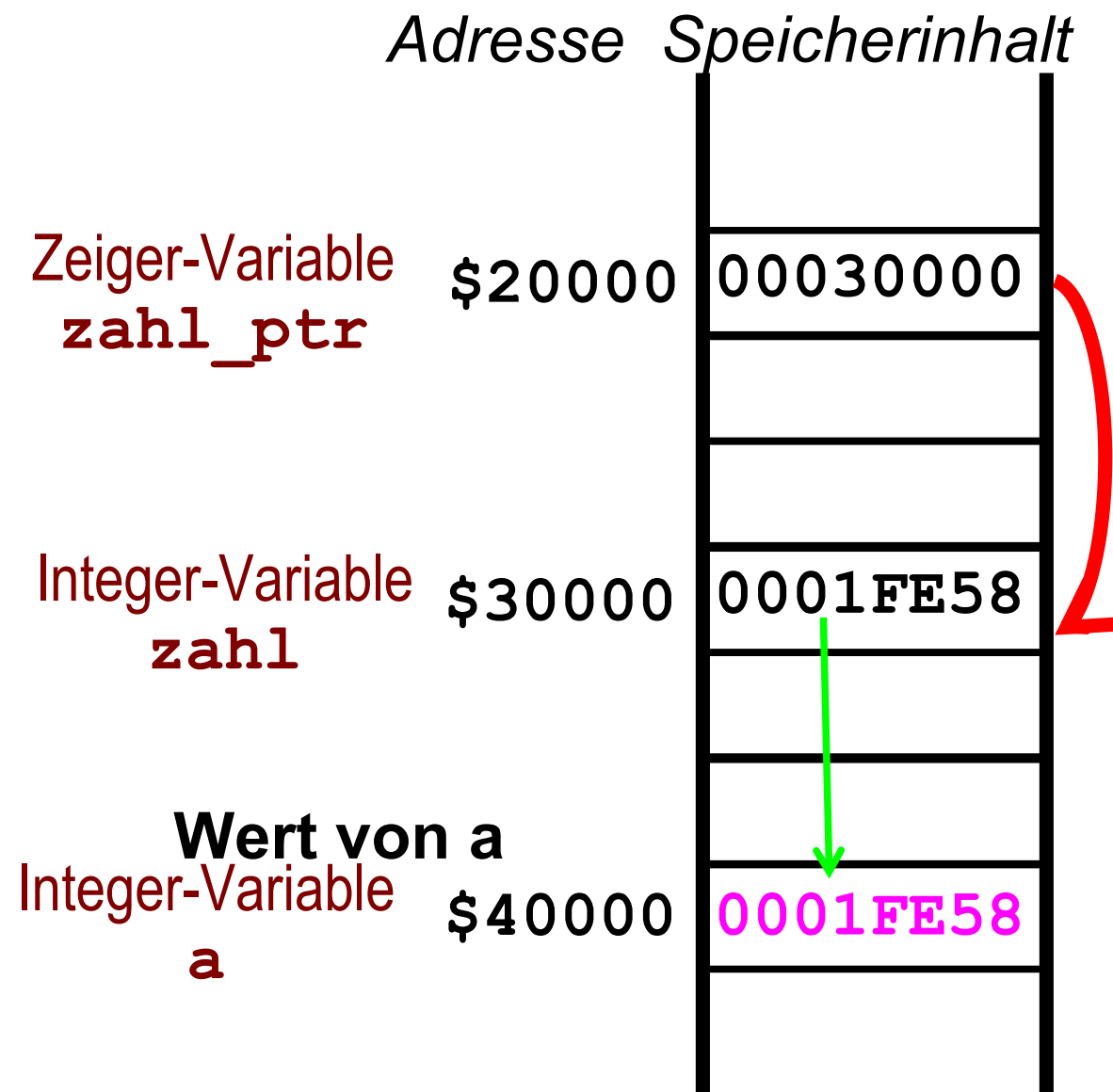
# Zeiger Beispiel 1

```
long int *zahl_ptr;  
long int zahl;  
long int a;
```

```
zahl = 0x0001FE58;  
zahl_ptr = &zahl;  
a = 0;
```

## Zuweisung

|                                           |                                |
|-------------------------------------------|--------------------------------|
| <code>a = zahl;</code>                    | <code>\$0001FE58</code>        |
| <code>a = (long int)&amp;zahl;</code>     | <code>\$00030000</code>        |
| <code>a = (long int)zahl_ptr;</code>      | <code>\$00030000</code>        |
| <code>a = (long int)&amp;zahl_ptr;</code> | <code>\$00020000</code>        |
| <u><code>a = *zahl_ptr;</code></u>        | <u><code>\$0001FE58</code></u> |



# Zeiger Beispiel 2

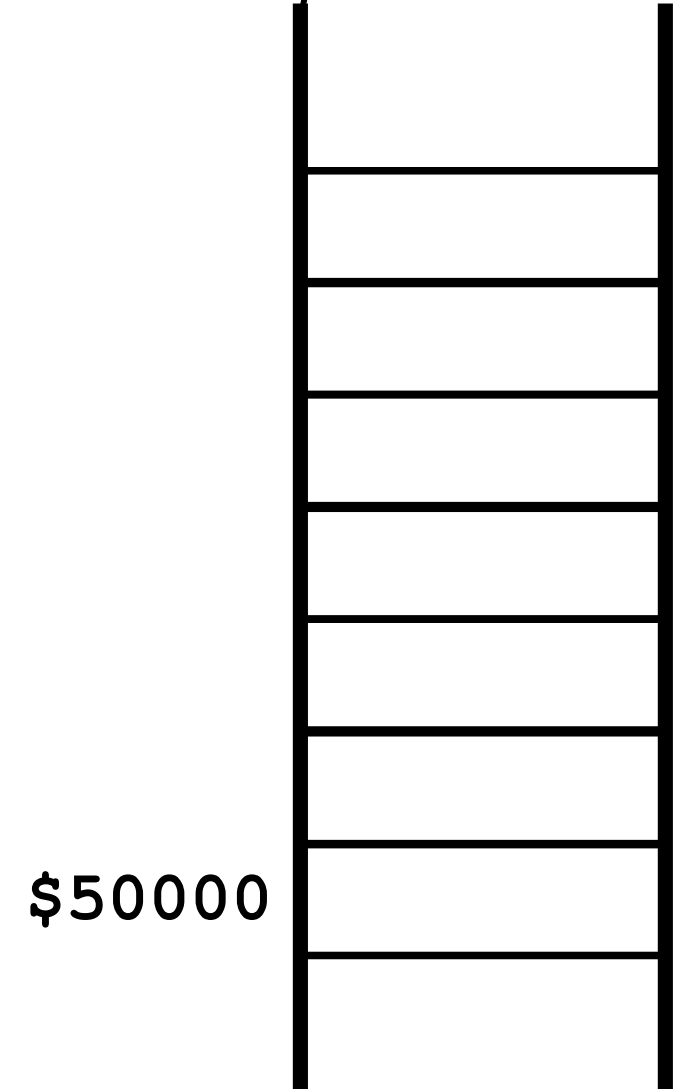
---

Zeiger auf absolute Adresse

```
#define    ADRESSE    0x00050000
...
long int   *abs_ptr;
...

// Zeiger initialisieren
abs_ptr = (long int *)ADRESSE;
...
// Zugriff auf absolute Adresse
*abs_ptr = (long int)10;
```

Adresse    Speicherinhalt

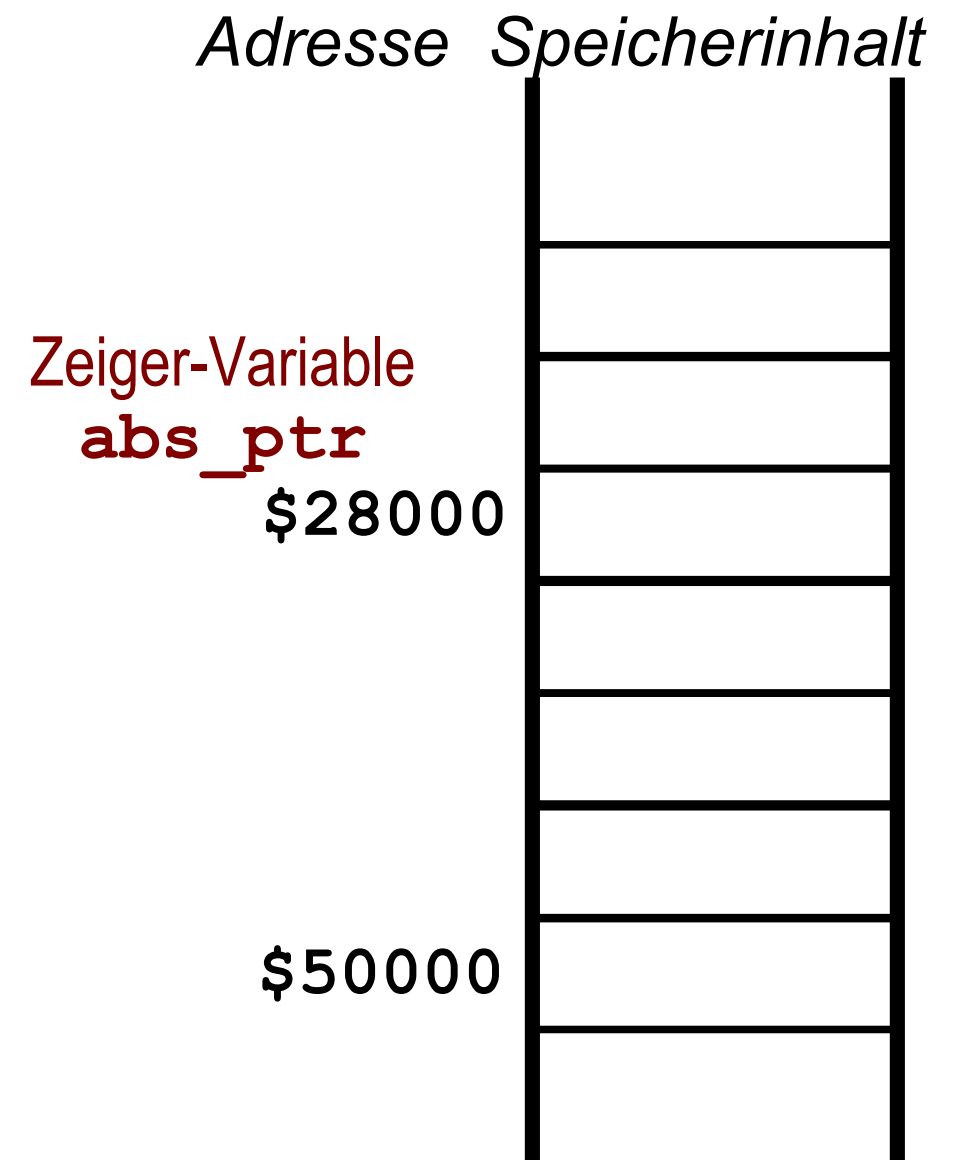


# Zeiger Beispiel 2

Zeiger auf absolute Adresse

```
#define    ADRESSE    0x00050000
...
long int  *abs_ptr;
...

// Zeiger initialisieren
abs_ptr = (long int *)ADRESSE;
...
// Zugriff auf absolute Adresse
*abs_ptr = (long int)10;
```

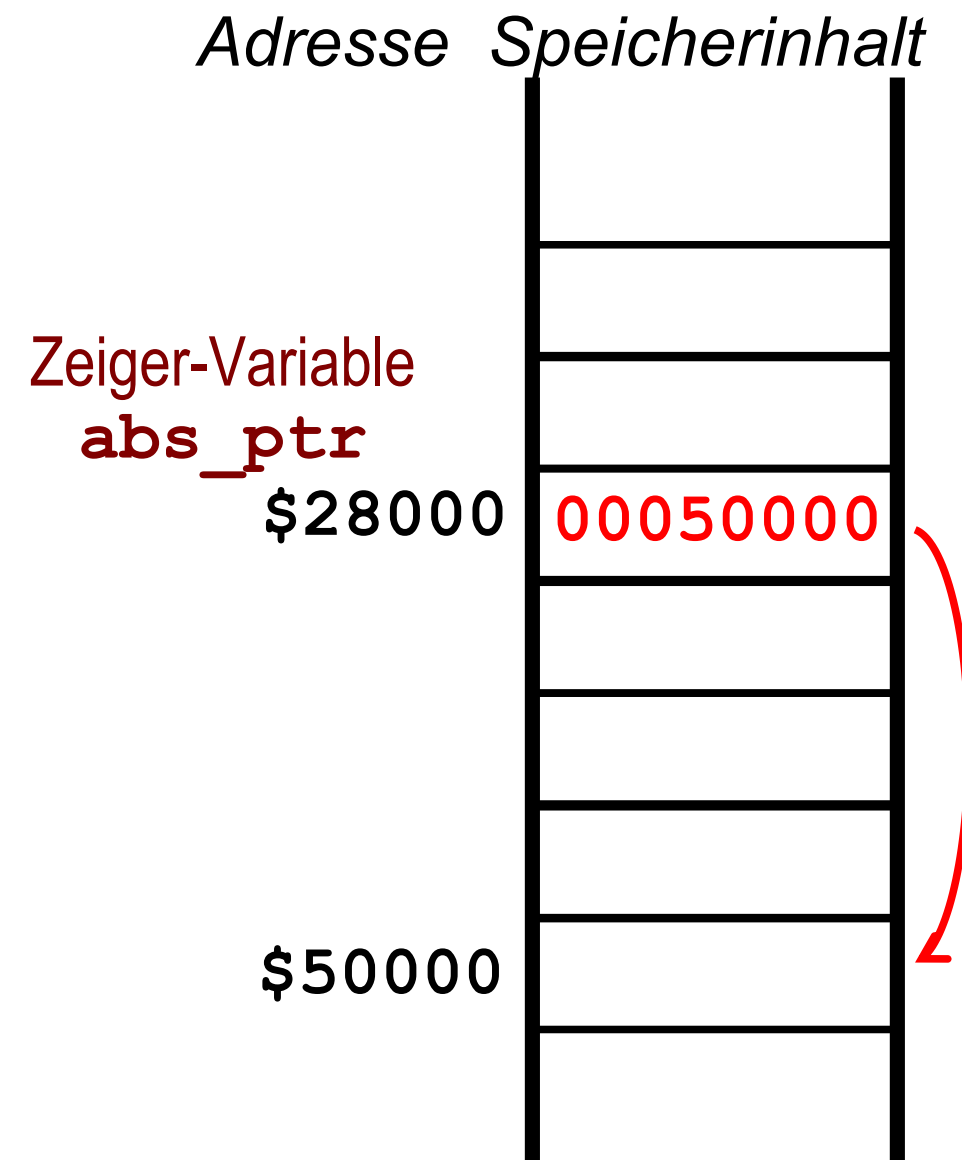


# Zeiger Beispiel 2

Zeiger auf absolute Adresse

```
#define    ADRESSE    0x00050000
...
long int  *abs_ptr;
...

// Zeiger initialisieren
abs_ptr = (long int *)ADRESSE;
...
// Zugriff auf absolute Adresse
*abs_ptr = (long int)10;
```



# Zeiger Beispiel 2

Zeiger auf absolute Adresse

```
#define    ADRESSE    0x00050000
...
long int  *abs_ptr;
...

// Zeiger initialisieren
abs_ptr = (long int *)ADRESSE;
...
// Zugriff auf absolute Adresse
*abs_ptr = (long int)10;
```

